Unit 4

INHERITANCE

Inheritance:

- Inheritance is a fundamental concept of Object-Oriented Programming (OOP) that allows a class (derived/child) to acquire properties (data members) and behaviors (member functions) from another class (base/parent).
- In simple words: “Inheritance means reusing an existing class to create a new class with extensions or modifications.”

Advantages:

- Code Reusability → Avoids rewriting common code.
- Extensibility → Extend or modify existing code easily.
- Improved Maintenance → Common features stay in base class, changes are centralized.
- Real-World Modeling → Represents IS-A relationship (e.g., Student IS-A Person).
- Supports Polymorphism → Enables function overriding and dynamic binding.

Syntax of Inheritance:

```
class Base {
    // members (data + functions)
};

class Derived : access_mode Base {
    // new members of derived class
};
```

- **Base class:** The existing class.
- **Derived class:** The new class created from base.
- **Access mode:** public, protected, or private → decides how base members are visible inside derived.

- Base class is created first → Derived automatically gets base data/functions.
- Constructors/Destructors: Base constructor runs before derived constructor; base destructor runs after derived destructor.
- Private members of base are not directly accessible in derived (need public/protected functions).
- Inheritance models IS-A relationship:
  Student is a Person
  Car is a Vehicle

1. private

- Members are accessible only within the class
- Default access level for class members

```
class Demo {
private:
    int a;  // only class functions can access this
};
```

2. public

Members are accessible from anywhere (including outside the class)

```
class Demo {
public:
    int x;  // accessible using object
};
```

3. protected

- Members are accessible within the class and its derived classes
- Not accessible from outside the class directly

```
class Base {
protected:
   int value;
};

class Derived : public Base {
public:
   void show() {
      cout << "Value: " << value;  // ✅ allowed in derived class
   }
};
```

Base member kind	With public inheritance	With protected inheritance	With private inheritance
`public`	public in derived	protected in derived	private in derived
`protected`	protected in derived	protected in derived	private in derived
`private`	not accessible	not accessible	not accessible

```cpp
#include <iostream>
using namespace std;

class Base {
public:    int pub = 1;
protected: int pro = 2;
private:   int pri = 3;
};

class DerPublic : public Base {
public:
    void testInside() {
        cout << "[DerPublic] pub=" << pub << ", pro=" << pro << "\n";
        // cout << pri; // ❌ private not accessible
    }
};

class DerProtected : protected Base {
public:
    void testInside() {
        // base public/protected become protected here
        cout << "[DerProtected] pub=" << pub << ", pro=" << pro << "\n";
    }
};

class DerPrivate : private Base {
public:
    void testInside() {
        // base public/protected become private here
        cout << "[DerPrivate] pub=" << pub << ", pro=" << pro << "\n";
    }
};

int main() {
    DerPublic dp;
    dp.testInside();
    cout << "Outside DerPublic: dp.pub=" << dp.pub << "\n";  // ✅ public remains public
    // cout << dp.pro; // ❌ protected

    DerProtected dprot;
    dprot.testInside();
    // cout << dprot.pub; // ❌ becomes protected outside
    // cout << dprot.pro; // ❌ protected

    DerPrivate dpriv;
    dpriv.testInside();
    // cout << dpriv.pub; // ❌ becomes private outside
    // cout << dpriv.pro; // ❌ private

    return 0;
}
```

```
[DerPublic] pub=1, pro=2
Outside DerPublic: dp.pub=1
[DerProtected] pub=1, pro=2
[DerPrivate] pub=1, pro=2
```

What this shows:

- public inheritance keeps pub accessible outside.
- protected inheritance hides base pub from outside (becomes protected).
- private inheritance hides both base pub and pro from outside (become private).

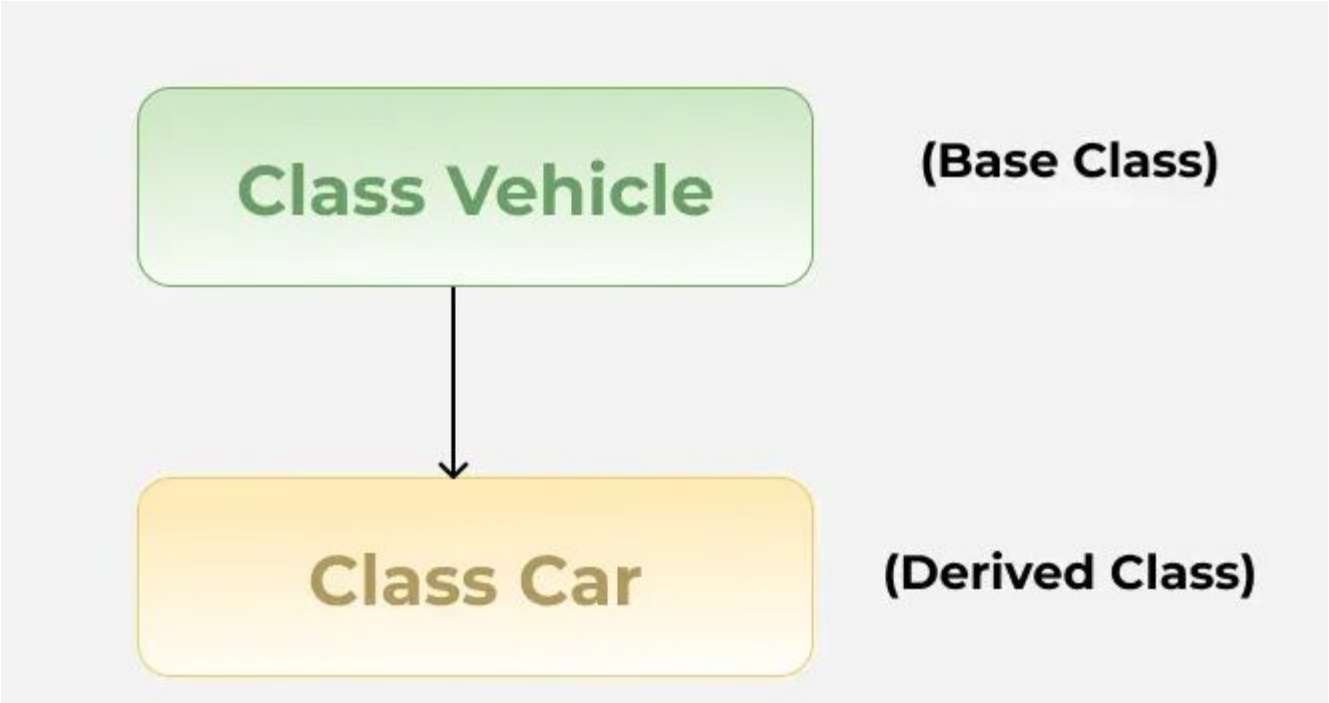
- **Types of Inheritance :**

Inheritance can be of 5 types in C++:

1. Single Inheritance
2. Multilevel Inheritance
3. Multiple Inheritance
4. Hierarchical Inheritance
5. Hybrid Inheritance

1 Single Inheritance:

- In Single Inheritance, a derived class inherits from only one base class.
- Base → Derived



- Example:

```
#include <iostream>
using namespace std;
class Animal {     // Base class
public:
    void eat() {
        cout << "This animal eats food." << endl;
    }
};

class Dog : public Animal {   // Derived class
public:
    void bark() {
        cout << "Dog barks!" << endl;
    }
};
int main() {
    Dog d;
    d.eat();   // From base class
    d.bark();  // From derived class
    return 0;
}
```

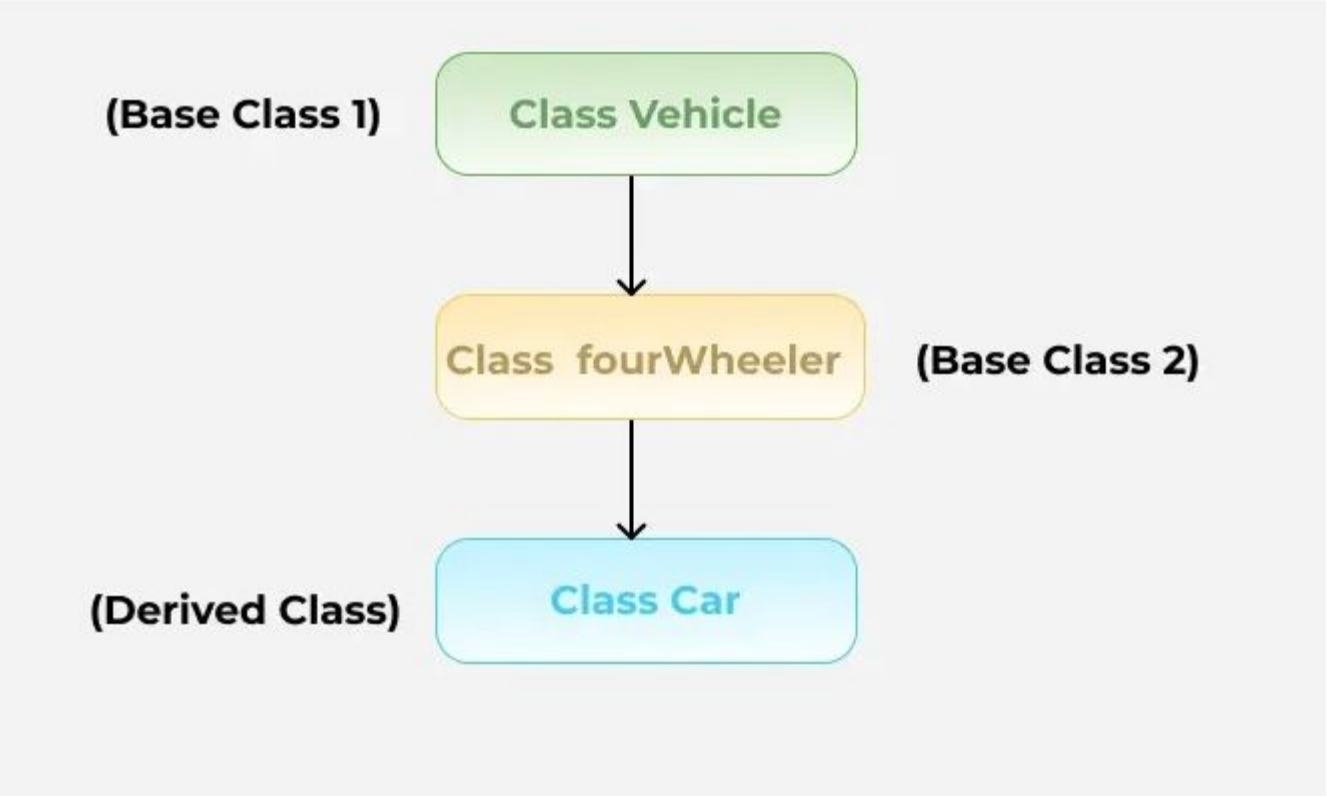
```
//Output

This animal eats food.
Dog barks!
```

2 Multilevel Inheritance:

In Multilevel Inheritance, a class is derived from another derived class (chain of inheritance).

Base → Derived1 → Derived2



```
#include <iostream>
using namespace std;
class GrandParent {
public:
    void house() {
        cout << "Grandparent's house." << endl;
    }
};
class Parent : public GrandParent {
public:
    void car() {
        cout << "Parent's car." << endl;
    }
};
class Child : public Parent {
public:
    void bike() {
        cout << "Child's bike." << endl;
    }
};
```

```
int main() {
    Child c;
    c.house(); // Inherited from
GrandParent
    c.car();   // Inherited from Parent
    c.bike();  // Child's own
    return 0;
}
```

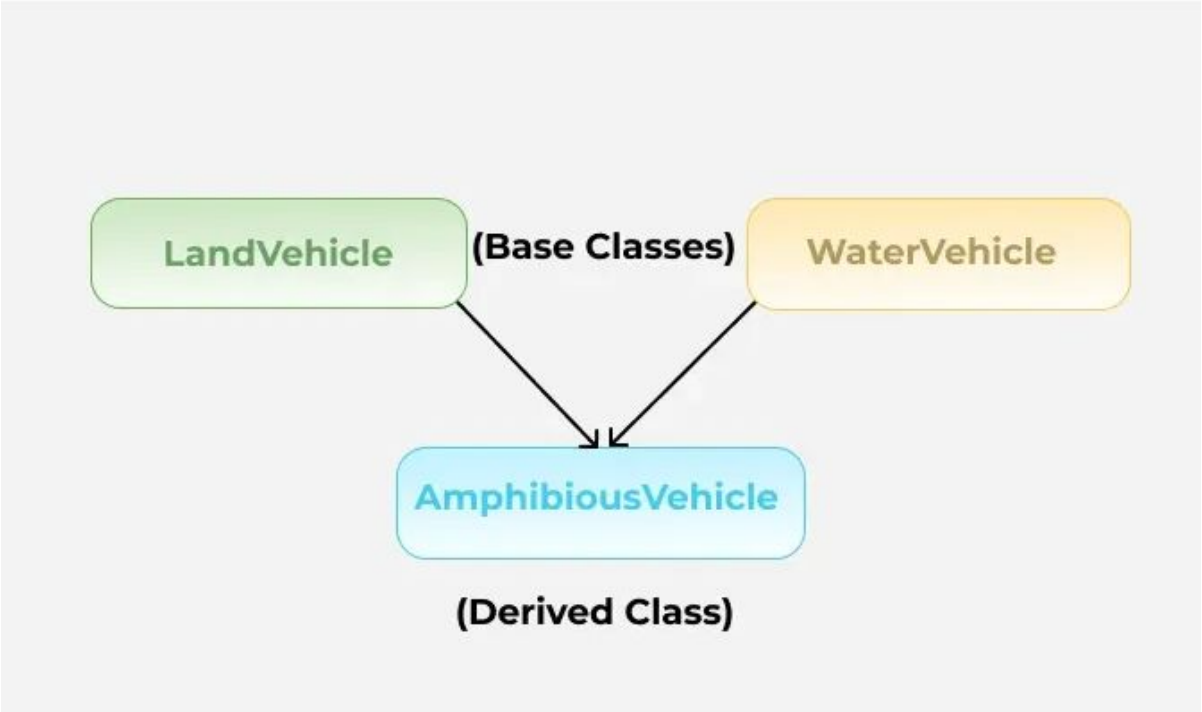
```
//Output

Grandparent's house.
Parent's car.
Child's bike.
```

3 Multiple Inheritance:

- In Multiple Inheritance, a class can inherit from more than one base class.

Base1 →
→ Derived
Base2 →



```cpp
#include <iostream>
using namespace std;
class Teacher {
public:
    void teach() {
        cout << "Teaching students." << endl;
    }
};
class Researcher {
public:
    void research() {
        cout << "Doing research work." << endl;
    }
};
class Professor : public Teacher, public Researcher {
public:
    void guide() {
        cout << "Guiding PhD students." << endl;
    }
};
int main() {
    Professor p;
    p.teach();     // From Teacher
    p.research();  // From Researcher
    p.guide();     // Professor's own
    return 0;
}
```

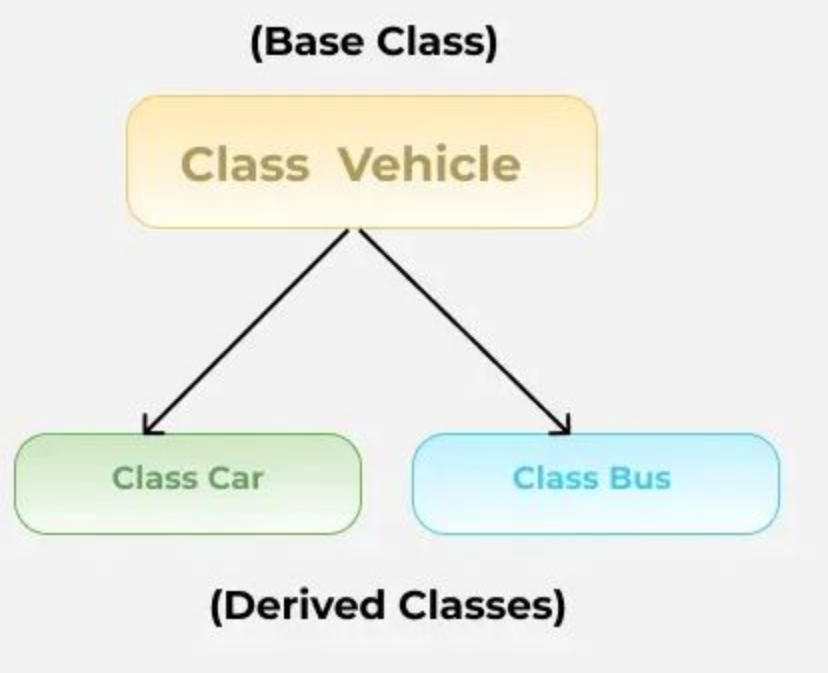
```
//Output

Teaching students.
Doing research work.
Guiding PhD students.
```

4 Hierarchical Inheritance:

- In Hierarchical Inheritance, multiple classes inherit from the same base class.

Base
/ \
Derived1 Derived2



```
#include <iostream>
using namespace std;
class Shape {
public:
    void draw() {
        cout << "Drawing shape." << endl;
    }
};
class Circle : public Shape {
public:
    void area() {
        cout << "Area of circle = πr²" << endl;
    }
};
class Rectangle : public Shape {
public:
    void area() {
        cout << "Area of rectangle = l × b" << endl;
    }
};
```

```
int main() {
    Circle c;
    Rectangle r;

    c.draw();
    c.area();

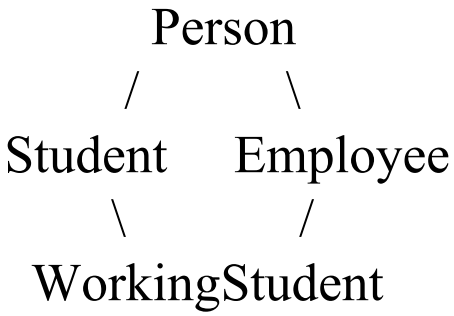
    r.draw();
    r.area();

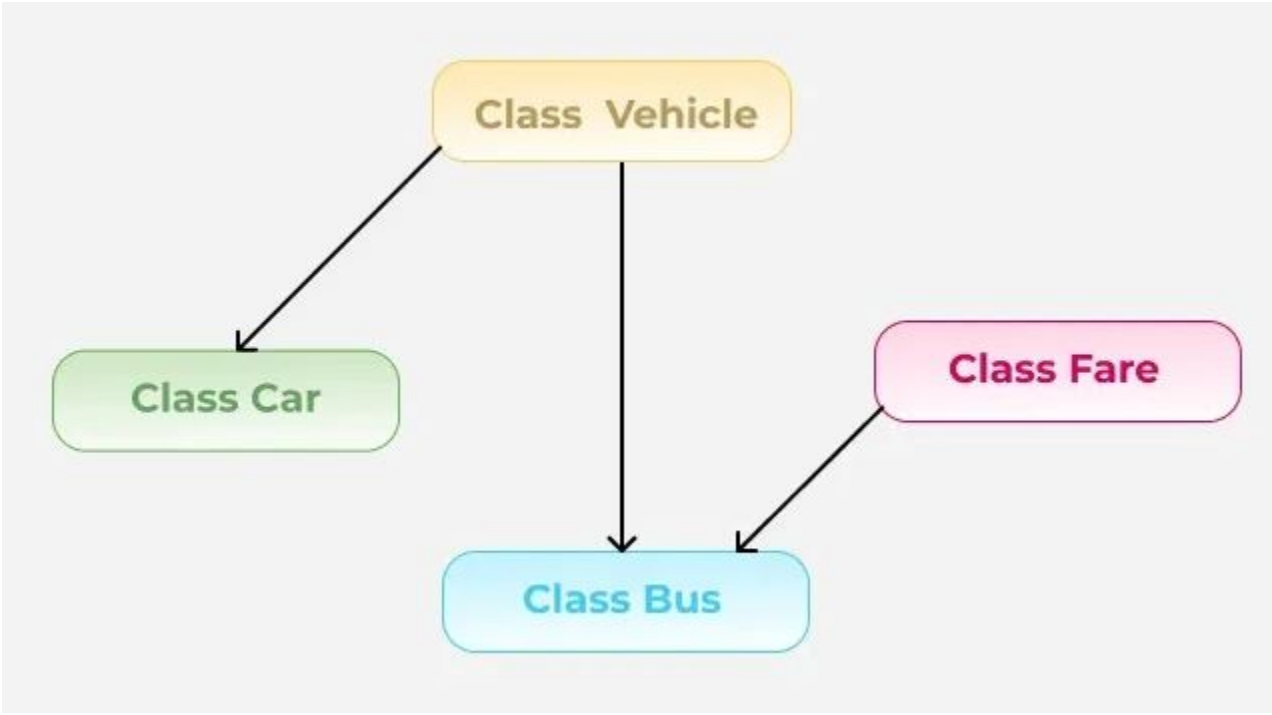
    return 0;
}
```

```
//Output
Drawing shape.
Area of circle = πr²
Drawing shape.
Area of rectangle = l × b
```

5 Hybrid Inheritance:

- Hybrid Inheritance = Combination of two or more types of inheritance.
- Commonly, it is Hierarchical + Multiple.
- In C++, virtual base classes are often used to solve ambiguity (Diamond Problem).





```
#include <iostream>
using namespace std;

class Person {
public:
    void showPerson() {
        cout << "I am a Person." << endl;
    }
};

class Student : virtual public Person {
public:
    void showStudent() {
        cout << "I am a Student." << endl;
    }
};

class Employee : virtual public Person {
public:
    void showEmployee() {
        cout << "I am an Employee." << endl;
    }
};

class WorkingStudent : public Student, public Employee {
public:
    void showWorkingStudent() {
        cout << "I am a Working Student." << endl;
    }
};
int main() {
    WorkingStudent ws;
    ws.showPerson();  // No ambiguity due to virtual inheritance
    ws.showStudent();
    ws.showEmployee();
    ws.showWorkingStudent();
    return 0;
}
```

```
//Output
I am a Person.
I am a Student.
I am an Employee.
I am a Working Student.
```

Type	Definition	Example Scenario
Single	One base → one derived	Animal → Dog
Multilevel	Chain of inheritance	Grandparent → Parent → Child
Multiple	Derived from >1 base	Teacher + Researcher → Professor
Hierarchical	One base → multiple derived	Shape → Circle, Rectangle
Hybrid	Combination of types (may cause diamond problem)	Person → Student + Employee → WorkingStudent

Virtual Base Classes :

Problem in Multiple Inheritance (Diamond Problem)

When a class is derived from two classes that themselves inherit from a common base class, ambiguity occurs because the derived class inherits multiple copies of the base class.

This is called the Diamond Problem.

```
class Person {
public:
    void showPerson() {
        cout << "I am a Person." << endl;
    }
};

class Student : public Person {
};

class Employee : public Person {
};

class WorkingStudent : public Student, public Employee {
};

int main() {
    WorkingStudent ws;
    // ws.showPerson(); // ❌ ERROR: Ambiguous (two copies of Person)
    return 0;
}
```

- **Here, WorkingStudent gets two copies of Person (one via Student, another via Employee).**
- **So, ws.showPerson() is ambiguous.**

- **Solution: Using virtual inheritance**
- **To avoid multiple copies of the base class, we use virtual base classes.**

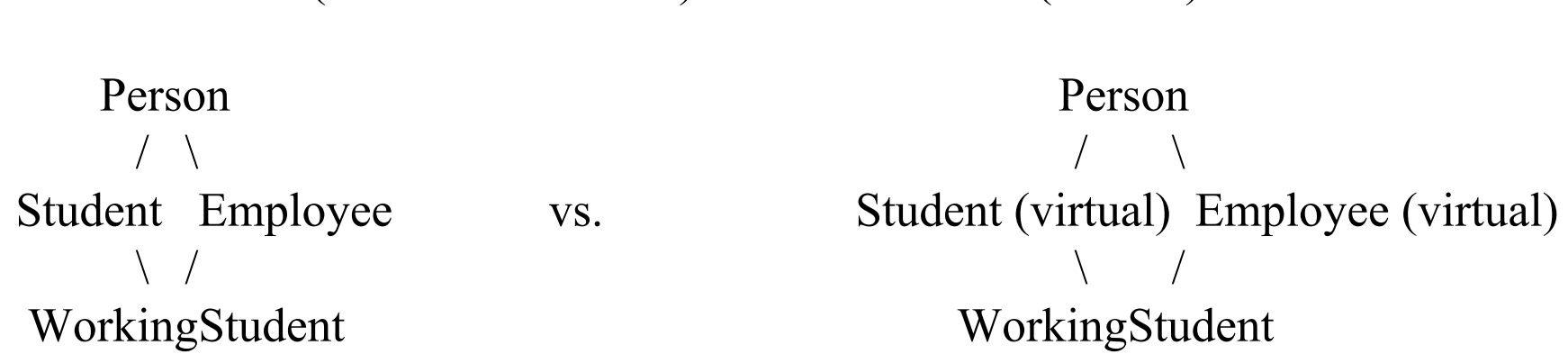
```
#include <iostream>
using namespace std;
class Person {
public:
    void showPerson() {
        cout << "I am a Person." << endl;
    }
};
// Inheriting virtually
class Student : virtual public Person {
};
class Employee : virtual public Person {
};
class WorkingStudent : public Student, public Employee {
};

int main() {
    WorkingStudent ws;
    ws.showPerson();  // ✅ No ambiguity, only one copy of Person exists
    return 0;
}
```

How it Works?

Student and Employee tell the compiler:
"If I am ever part of a multiple inheritance, don't create a separate copy of Person. Share the same one."

So, WorkingStudent now has only one shared copy of Person.



- Prevents multiple copies of a common base class.
- Used to resolve ambiguity in multiple inheritance.
- Saves memory by sharing a single base class object.

Abstract Classes in C++:

Definition:

- An abstract class in C++ is a class that cannot be instantiated (i.e., you cannot create objects of it directly).
- It is designed to be a base class that other classes inherit from.
- An abstract class is created by declaring at least one pure virtual function.

Pure Virtual Function:

- A pure virtual function is a virtual function with no implementation in the base class.
- Its syntax:

```
virtual void functionName() = 0;
```

Here, = 0 tells the compiler that this function must be overridden in the derived class.

Key Features:

- Cannot create objects of an abstract class.
- Can have constructors, destructors, and even normal functions.
- Used to provide a common interface to derived classes.
- A class becomes abstract if it has at least one pure virtual function.
- Derived class must override all pure virtual functions, otherwise it also becomes abstract.

```cpp
// Abstract Base Class
class Shape {
public:
    // Pure Virtual Function
    virtual void draw() = 0;

    // Normal Function
    void info() {
        cout << "This is a shape." << endl;
    }
};

// Derived Class 1
class Circle : public Shape {
public:
    void draw() override {
        cout << "Drawing a Circle." << endl;
    }
};

// Derived Class 2
class Rectangle : public Shape {
public:
    void draw() override {
        cout << "Drawing a Rectangle." << endl;
```

```cpp
int main() {
    // Shape s; ❌ Error: cannot create
object of abstract class

    Circle c;
    Rectangle r;

    Shape *ptr;  // Pointer to abstract class

    // Pointing to Circle
    ptr = &c;
    ptr->draw();   // ✅ Drawing a Circle

    // Pointing to Rectangle
    ptr = &r;
    ptr->draw();   // ✅ Drawing a Rectangle

    return 0;
}

// Output
Drawing a Circle.
Drawing a Rectangle.
```

- Shape is an abstract class because it has a pure virtual function draw().
- You cannot create an object of Shape.
- Circle and Rectangle override the draw() function → so they are concrete classes.
- Using a base class pointer (Shape*), we can call the correct version of draw() at runtime (runtime polymorphism).
- Abstract classes are mainly used when you want to define a common interface (like Shape) but leave the actual implementation details (like draw()) to the derived classes.

Use an Abstract Class :

To provide a common interface
Abstract classes allow you to define a blueprint (set of pure virtual functions) that derived classes must implement.
Example: A Shape abstract class with a pure virtual function draw() ensures every subclass (Circle, Rectangle, Triangle) defines its own way of drawing.

To achieve runtime polymorphism
Abstract classes enable dynamic (runtime) polymorphism. You can use base-class pointers or references to call derived-class implementations.
This is essential in cases where the exact type of object is not known at compile time.

To enforce implementation rules
By declaring a pure virtual function, you force derived classes to implement that function.
This guarantees that all subclasses follow a consistent interface.

Code reusability + specialization
Abstract class can contain both:
Common code (normal member functions, data members).
Pure virtual functions (to be implemented differently by each subclass).
This avoids code duplication while still allowing specialization.

Design clarity
Abstract classes clearly separate "what should be done" (interface) from "how it is done" (implementation in derived classes).
This makes large systems easier to design, read, and maintain.

Constructors in Derived Classes:

Concept:
When we create an object of a derived class, the constructor of the base class is called first, and then the constructor of the derived class is called.
This ensures that the base part of the object is properly initialized before the derived part.

Order of execution:
Base class constructor → Derived class constructor

Order of destruction:
Derived class destructor → Base class destructor

Key Points:

- If the base class has a default constructor, the compiler automatically calls it.
- If the base class has a parameterized constructor, the derived class must explicitly call it in its constructor using the initializer list.
- Derived classes can have their own constructors, which may or may not take parameters.

```
class Base {
public:
    Base() {
        cout << "Base class constructor called" << endl;
    }
};

class Derived : public Base {
public:
    Derived() {
        cout << "Derived class constructor called" << endl;
    }
};

int main() {
    Derived d;   // Create object of derived class
    return 0;
}
```

✅ Output

```
Base class constructor called
Derived class constructor called
```

***First Base() is called, then Derived().**

Example 2: Parameterized Constructor in Base and Derived

```
#include <iostream>
using namespace std;
class Base {
    int x;
public:
    Base(int a) {   // Parameterized constructor
        x = a;
        cout << "Base class constructor called, x = " << x << endl;
    }
};

class Derived : public Base {
    int y;
public:
    Derived(int a, int b) : Base(a) {  // Call base constructor explicitly
        y = b;
        cout << "Derived class constructor called, y = " << y << endl;
    }
};
int main() {
    Derived d(10, 20);
    return 0;
}
```

```
//Output
Base class constructor called, x = 10
Derived class constructor called, y = 20
```

Notice how we used : Base(a) to call the base class constructor with an argument.

Example 3: Multiple Inheritance (More than One Base Class)

```
#include <iostream>
using namespace std;
class Base1 {
public:
    Base1() {
        cout << "Base1 constructor" << endl;
    }
};
class Base2 {
public:
    Base2() {
        cout << "Base2 constructor" << endl;
    }
};
class Derived : public Base1, public Base2 {
public:
    Derived() {
        cout << "Derived constructor" << endl;
    }
};

int main() {
    Derived d;
    return 0;
```

```
//Output
Base1 constructor
Base2 constructor
Derived constructor
```

Constructors are called in the order of inheritance (left to right).

Example 4: Destructor Order

```
#include <iostream>
using namespace std;
class Base {
public:
   Base() {
      cout << "Base constructor" << endl;
   }
   ~Base() {
      cout << "Base destructor" << endl;
   }
};
class Derived : public Base {
public:
   Derived() {
      cout << "Derived constructor" << endl;
   }
   ~Derived() {
      cout << "Derived destructor" << endl;
   }
};

int main() {
   Derived d;
   return 0;
```

```
// Output
Base constructor
Derived constructor
Derived destructor
Base destructor
```

Destructors are called in reverse order of constructors.

- Base constructor always executes before derived constructor.
- If base has a parameterized constructor → derived must call it explicitly.
- With multiple inheritance → base constructors are called left to right.
- Destructors execute in reverse order.